

[7. REST](#)

[7.1 REST - základné pojmy](#)

[7.2 Prvá REST aplikácia](#)

[7.3 REST služba s viacerými zdrojmi](#)

[Cvičenia](#)

7. REST

REST – **Representational state transfer** – je softvérový architektonický štýl navrhnutý pre vývoj architektúry pre World Wide Web. Kladie dôraz na škálovateľnosť interakcií medzi komponentmi, jednotné rozhrania, nezávislý vývoj a nasadenie komponentov. Definuje súbor obmedzení (požiadaviek, zásad a princípov) kladených na architektúru zodpovedajúcu paradigme REST:

- **Klient-server** – klient a server sú nezávislé.
- **Bezstavový server** – nezaznamenáva stav klienta.
- **Využitie medzipamäte** – server označí dáta, ktoré možno ukladať do medzipamäte.
- **Jednotné rozhranie**
- **Viacvrstvový systém** – komponenty sprostredkujúce komunikáciu medzi klientom a serverom správanie neovplyvňujú.

7.1 REST - základné pojmy

Hoci REST bol pôvodne navrhnutý ako súbor princípov pre architektúru webu, má v súčasnosti oveľa širšie uplatnenie, najmä pri návrhu komunikačných rozhraní v distribuovaných aplikáciách a systémoch založených na webových službách.

Komunikačné rozhranie služby, navrhnuté na základe požiadaviek architektúry REST, sa označuje REST-API. V tomto kontexte je popri bezstavovej klient-server komunikácii kľúčovou požiadavka jednotného rozhrania.

Základné koncepty REST-API

- **Zdroj** (resource) – kľúčová abstrakcia. Zdrojom môže byť akýkoľvek objekt nachádzajúci sa na webe, ktorý je jednoznačne identifikovaný svojím URI.
- **Klient-server komunikácia** – Klient komunikuje so serverom pomocou správ (požiadavky a odpovede), ktoré obsahujú reprezentáciu stavu zdroja. (Odtiaľ je odvodená skratka REST = representational state transfer.)
- **Reprezentácia** je dokument, ktorý zachytáva aktuálny alebo nový stav objektu. Typicky to je HTML, XML, JSON, prípadne jednoduchý textový dokument. Môže to však byť akýkoľvek súbor – obrázok, pdf, zip ...
- **Formát reprezentácie** v správe udáva Internet Media Type (skrátene MIME-typ).
- **Univerzálne rozhranie** – REST-API poskytuje na manipuláciu so zdrojmi **jednotnú sadu CRUD operácií**.

RESTful web service

Webová služba založená na princípoch REST poskytuje REST-API na prístup ku kolekcií zdrojov, ktoré spravuje. Zdroje typicky tvoria hierarchickú štruktúru, ktorú odzrkadľuje ich URL.

Príklad - Zdroj s URI:

- `http://moja.skola.sk/resources/predmety`

- je koreň hierarchie zdrojov s informáciami o predmetoch,
- <http://moja.skola.sk/resources/predmety/VSA>
je URI zdroja s informáciami k predmetu VSA a
- <http://moja.skola.sk/resources/predmety/VSA/P1>
je URI dokumentu k 1. prednáške z predmetu VSA.

CRUD operácie pre manipuláciu so zdrojmi sú priamo namapované na požiadavky protokolu HTTP:

Operácia / Typ zdroja	GET	PUT	POST	DELETE
kolekcia zdrojov	Vráti vhodnú reprezentáciu zoznamu zdrojov, ktorá umožní prístup k jednotlivým zdrojom v kolekcii. Např. zoznam relatívnych URI zdrojov v kolekcii.	Nahradí celú kolekciu novou. (Menej používané)	Pridá nový zdroj do kolekcie zdrojov	odstráni celú kolekciu zdrojov
koncový zdroj v hierarchii	Vráti reprezentáciu zdroja v požadovanom formáte (media type).	Aktualizuje stav zdroja resp. nahradí starý stav novým. Idempotentná metóda.	Považuje zdroj za kolekciu a vloží do nej nový podzdroj. (Menej používané)	odstráni zdroj

JAX-RS

Súčasťou platformy JakartaEE je knižnica JAX-RS implementujúca podporu pre vývoj REST služieb v jazyku Java. Pozri např.

- <https://jakarta.ee/learn/starter-guides/how-to-build-a-restful-web-service/>
- <https://eclipse-ee4j.github.io/jakartaee-tutorial/>

Knižnica poskytuje vývojárom predovšetkým sadu anotácií, umožňujúcich:

- Definovať URI zdrojov spravovaných REST službou.
- Prepojiť HTTP požiadavky s metódami zdroja, implementujúcimi CRUD operácie.
- Extrahovať údaje z HTTP požiadaviek a poskytnúť ich ako argumenty metódam zdroja.

REST služba

Implementácia (serverovej strany) REST služby vytvorená pomocou JAX-RS pozostáva z jednej alebo viacerých **tried zdroja** a prípadne ďalších podporných tried.

Trieda zdroja predstavuje prístupový bod ku zdroju, resp. kolekcií zdrojov. Trieda je označená anotáciou `@Path`, ktorá definuje relatívnu URI zdroja. Ak trieda sprístupňuje celú kolekciu zdrojov, je to URI koreňového zdroja v hierarchii zdrojov.

Metódy zdroja - Trieda implementuje CRUD metódy na prístup a manipuláciu so zdrojmi. Tieto sú namapované na HTTP požiadavky pomocou anotácií `@GET`, `@PUT`, `@POST`, `@DELETE` ...

Formát správ (MIME type - Internet media type) - Metódy zdroja, ktoré posielajú obsah klientovi (GET, príp. POST), špecifikujú jeho formát anotáciou `@Produces`. Metódy zdroja, ktoré prijímajú obsah od klienta (PUT, POST) špecifikujú anotáciou `@Consumes`, aký formát obsahu akceptujú.

REST-aplikácia

Aplikáciu poskytujúcu REST službu (t.j. implementujúcu serverovú stranu služby) budeme nazývať **REST-aplikácia**. REST-aplikácia môže byť realizovaná dvoma spôsobmi:

- **ako súčasť webovej aplikácie** - Služba nie je samostatne spustiteľná aplikácia ale iba komponent (tzv. war-súbor), ktorý sa nasadzuje a spúšťa na webovom serveri. Server je kontajner, ktorý poskytuje infraštruktúru na komunikáciu prostredníctvom protokolu HTTP.

- **ako samostatná aplikácia** (standalone). V tomto prípade musí byť infraštruktúra na komunikáciu prostredníctvom protokolu HTTP integrovaná priamo v aplikácii.

7.2 Prvá REST aplikácia

Príklad 1 - REST služba s jedným zdrojom

Trieda zdroja `SimpleResource` poskytuje klientom prístup k jednému zdroju prostredníctvom textových správ.

```
@Path("message")
@Singleton
public class SimpleResource {
    private String oznam = "ziadny oznam";

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getMessage() {
        return oznam;
    }

    @PUT
    @Consumes(MediaType.TEXT_PLAIN)
    public void putMessage(String content) {
        oznam = content;
    }
}
```

Zdroj predstavuje jednoduchý textový oznam. Jeho relatívna URI je definovaná anotáciou triedy `@Path("message")`.

Pre prístup k zdroju - oznamu - implementuje trieda dve metódy:

- Anotácia `@GET` označuje metódu `getMessage`, ktorá je vyvolaná po prijatí HTTP požiadavky GET. Návratová hodnota - obsah oznamu - sa pošle ako odpoveď klientovi. Anotácia `@Produces(MediaType.TEXT_PLAIN)` hovorí, že odpoveď bude jednoduchý text.
- Obsah oznamu môžu klienti zmeniť pomocou HTTP požiadavky PUT. Táto požiadavka bude spracovaná metódou `putMessage` označenou anotáciou `@PUT`. Anotácia `@Consumes(MediaType.TEXT_PLAIN)` hovorí, že metóda akceptuje od klientov požiadavky s jednoduchým textovým obsahom.

Stav zdroja - aktuálny obsah oznamu - uchováваме zatiaľ len v pamäti ako dátový člen triedy zdroja. Anotácia triedy zdroja `@Singleton` zabezpečí, že sa vytvorí len jedna inštancia triedy zdroja, ktorá spracuje všetky HTTP požiadavky na zdroj počas celého behu aplikácie. Bez tejto anotácie by sa pre obsluhu každej požiadavky vytvárala vždy nová inštancia triedy zdroja - oznam by bol teda vždy nanovo inicializovaný. Pozri cvičenie 7.2.

Poznámka: Anotácia `@Singleton` sa nachádza v balíku `jakarta.inject`.

Podobnú funkcionálnu možnosť možno dosiahnuť aj inými anotáciami, napr. `@Stateless` z balíka `jakarta.ejb` alebo `@ApplicationScoped` z balíka `jakarta.enterprise.context`.

Implementácia služby ako samostatná aplikácia

Pre implementáciu REST služby ako samostatne spustiteľnej aplikácie stačí vytvoriť jednoduchý **projekt aplikácie**. Do pom súboru projektu je potrebné pridať závislosti pre komponenty poskytujúce infraštruktúru na komunikáciu prostredníctvom protokolu HTTP a implementáciu služby.

```
<dependencies>
```

```

<dependency>
  <groupId>org.glassfish.jersey.inject</groupId>
  <artifactId>jersey-hk2</artifactId>
  <version>3.1.9</version>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-moxy</artifactId>
  <version>3.1.9</version>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-jaxb</artifactId>
  <version>3.1.9</version>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-json-binding</artifactId>
  <version>3.1.9</version>
</dependency>

<!-- JAXB -->
<dependency>
  <groupId>jakarta.xml.bind</groupId>
  <artifactId>jakarta.xml.bind-api</artifactId>
  <version>3.0.1</version>
</dependency>

<!-- Grizzly embedded server -->
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-grizzly2-http</artifactId>
  <version>3.1.9</version>
</dependency>
</dependencies>

```

Komunikáciu prostredníctvom protokolu HTTP bude zabezpečovať embedovaný Grizzly `HttpServer`. Na jeho konfiguráciu a spustenie môžeme implementovať jednoduchú metódu:

```

public static final String BASE_URI = "http://localhost:8080/SimpleRestApp";

static HttpServer startServer() {
    // create a resource config that scans for JAX-RS resources in rest package
    final ResourceConfig rc = new ResourceConfig().packages("rest");

    // create and start a new instance of grizzly http server
    // exposing the Jersey application at BASE_URI
    return GrizzlyHttpServerFactory.createHttpServer(URI.create(BASE_URI), rc);
}

```

Najprv vytvorí konfiguráciu `ResourceConfig().packages("rest")` s menom balíka, v ktorom sa nachádzajú triedy zdrojov. Následne vytvorí inštanciu servera, pričom nastaví základnú URI zdrojov `http://localhost:8080/SimpleRestApp`.

Server môžeme spustiť priamo v main metóde aplikácie a tam ho aj následne zastaviť:

```

public static void main(String[] args) throws IOException {
    final HttpServer server = startServer();
    System.out.println("Jersey app started with endpoints at "+BASE_URI);
    System.out.println("Hit Ctrl-C to stop it...");

    System.in.read();
    server.stop();
}

```

Službu z príkladu 1 implementovanú ako samostatnú aplikáciu nájdete v projekte [SimpleRestApp](#).

Po spustení aplikácie môžeme REST službu otestovať aj priamo prehliadačom odoslaním GET požiadavky na URI oznamu <http://localhost:8080/SimpleRestApp/message>

REST klient

Klientom REST služby môže byť akákoľvek aplikácia, ktorá dokáže posielať HTTP požiadavky a prijímať odpovede. Knižnica JAX-RS má takisto podporu pre vytvorenie REST klienta v jazyku Java. Do pom súboru je potrebné pridať rovnaké závislosti ako pre implementáciu služby (ako samostatnej aplikácie - pozri vyššie) ale namiesto embedovaného Grizzly servera treba pridať závislosť pre implementáciu klienta:

```
<dependencies>
  <dependency>
    <groupId>org.glassfish.jersey.inject</groupId>
    <artifactId>jersey-hk2</artifactId>
    <version>3.1.9</version>
  </dependency>
  <dependency>
    <groupId>org.glassfish.jersey.media</groupId>
    <artifactId>jersey-media-moxy</artifactId>
    <version>3.1.9</version>
  </dependency>
  <dependency>
    <groupId>org.glassfish.jersey.media</groupId>
    <artifactId>jersey-media-jaxb</artifactId>
    <version>3.1.9</version>
  </dependency>
  <dependency>
    <groupId>org.glassfish.jersey.media</groupId>
    <artifactId>jersey-media-json-binding</artifactId>
    <version>3.1.9</version>
  </dependency>

  <!-- JAXB -->
  <dependency>
    <groupId>jakarta.xml.bind</groupId>
    <artifactId>jakarta.xml.bind-api</artifactId>
    <version>3.0.1</version>
  </dependency>

  <!-- Jersey client -->
  <dependency>
    <groupId>org.glassfish.jersey.core</groupId>
    <artifactId>jersey-client</artifactId>
    <version>3.1.9</version>
  </dependency>
</dependencies>
```

Podporu pre implementáciu klienta služby poskytuje trieda `WebTarget`. Jej inštanciu vytvoríme pomocou triedy `ClientBuilder`, pričom definujeme URI zdroja. Následne ju môžeme použiť na prácu so zdrojom prostredníctvom HTTP požiadaviek.

V nasledujúcej ukážku klient služby z príkladu 1 najprv požiadavkou PUT zmení obsah oznamu a následne si aktualizovaný stav požiadavkou GET načíta.

```
String BASE_URI = "http://localhost:8080/SimpleRestApp";

Client client = ClientBuilder.newClient();
WebTarget webTarget = client.target(BASE_URI).path("message");

// putting message to rest service
String message = "Hello";
webTarget.request(MediaType.TEXT_PLAIN)
    .put(Entity.entity(message, MediaType.TEXT_PLAIN));
```

```
// getting message from rest service
String result = webTarget.request(MediaType.TEXT_PLAIN).get(String.class);
System.out.println("GET returned: " + result);
```

Implementáciu tohto klienta nájdete v projekte [SimpleRestClient](#).

Implementácia služby ako súčasť webovej aplikácie

Pre implementáciu REST služby ako súčasť webovej aplikácie je potrebné vytvoriť **projekt web aplikácie**. Súbor pom by už mal obsahovať závislosti na komponentoch, ktoré pre web aplikácie poskytuje cieľový aplikačný server (napr. jakarta.jakartaee-api). Do závislostí je vhodné pridať aj komponentu JAXB, ktorá nám umožňuje serializovať dátové objekty do XML:

```
<dependency>
  <groupId>jakarta.platform</groupId>
  <artifactId>jakarta.jakartaee-api</artifactId>
  <version>${jakartaee}</version>
  <scope>provided</scope>
</dependency>
<!-- JAXB -->
<dependency>
  <groupId>jakarta.xml.bind</groupId>
  <artifactId>jakarta.xml.bind-api</artifactId>
  <version>3.0.1</version>
</dependency>
```

Webová aplikácia poskytujúca REST službu musí mať okrem **triedy zdrojov** aj špeciálnu **konfiguračnú triedu web aplikácie** odvodenú od `jakarta.ws.rs.core.Application`, ktorá zabezpečuje registráciu REST služby:

```
import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;
@ApplicationPath("resources")
public class JakartaRestConfiguration extends Application { }
```

Anotácia triedy `@ApplicationPath("resources")` definuje relatívnu URI, ktorá slúži ako **bázové URI** všetkých spravovaných zdrojov. Úplná URI zdroja oznamu z príkladu 1 by teraz bola:

```
<ip-servera>/<aplikacia>/resources/message
```

kde

- `<ip-servera>` je ip adresa webového servera, na ktorom je služba nasadená
- `<aplikacia>` je názov webovej aplikácie, ktorá implementuje službu (typicky zhodný s názvom projektu).

Poznámky:

- Prostredie **apache netbeans 23** je možné integrovať s aplikačným serverom **Glassfish 8**, a testovať na ňom webové aplikácie a služby.
- Projekt webovej aplikácie vytvorený v prostredí apache netbeans nástrojom **New project...→Java with Maven→Web Application** už obsahuje konfiguračnú triedu web aplikácie a .pom súbor so závislosťou `jakartaee-api` vo verzii pre tento server.

Upozornenie: Prostredie apache netbeans poskytuje tiež integrovaných klientov na testovanie REST služieb a nástroje, na ktoré uľahčujú tvorbu klientských java aplikácií. Niektoré z týchto nástrojov však majú isté nedostatky (a aj drobné bugy).

7.3 REST služba s viacerými zdrojmi

Jedna REST služba môže poskytovať prístup k mnohým zdrojom prostredníctvom jednej alebo viacerých tried zdroja. Každá trieda môže implementovať prístupové metódy pre celú kolekciu zdrojov, ktoré sú hierarchicky usporiadané.

V nasledujúcom príklade ilustrujeme využitie základných JAX-RS anotácií pri implementácii REST

služby s viacerými zdrojmi.

Príklad 2: REST služba školy

Príklad ilustruje triedu zdroja, ktorá implementuje prístupové metódy pre kolekciu zdrojov reprezentujúcich informácie o škole a vyučovacích predmetoch.

```
@Path("skola")
public class SkolaResource {
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getInfo() {
        return "FEI STU";
    }

    @GET
    @Path("adresa")
    @Produces(MediaType.TEXT_PLAIN)
    public String getAdresa() {
        return "Ilkovicova 2, Bratislava";
    }
}
```

Anotácia `@Path("skola")` triedy zdroja definuje relatívnu URI koreňového zdroja v hierarchii.

Požiadavka GET na koreň vráti názov školy. Adresa školy predstavuje samostatný podzdroj.

Relatívna URI podzdroja je definovaná anotáciou prístupovej metódy `@Path("adresa")`.

Teda URI adresy školy je:

```
<ip-servera>/<aplikacia>/resources/skola/adresa.
```

Fragmenty URI zdroja sú parametrizovateľné. Jedna metóda môže tak realizovať prístup k viacerým zdrojom. Informácie o predmetoch môže poskytovať nasledujúca metóda.

```
@GET
@Produces(MediaType.TEXT_PLAIN)
@Path("predmet/{nazov}")
public String getPredmet(@PathParam("nazov") String name) {
    if ("VSA".equals(name)) return "Informácie k predmetu VSA ...";
    if ("OOP".equals(name)) return "Informácie k predmetu OOP ...";
    // a.t.d.
    return null;
}
```

Parametrizovateľný fragment URI v anotácii `@Path("predmet/{nazov}")` je uzavretý v zložených zátvorkách. Jeho hodnota, názov predmetu, je anotáciou `@PathParam` injektovaná ako argument metódy. Informácie o predmete VSA budú teda prístupné pod URI:

```
<ip-servera>/<aplikacia>/resources/skola/predmet/VSA
```

Trieda Response

Keď metóda zdroja vráti hodnotu `null`, klientovi sa odošle odpoveď so status kódom 404-Not Found. Ak by sme chceli poslať odpoveď s iným kódom, napr. 204-No Content, je potrebné, aby metóda vracala inštanciu triedy **Response**. Táto trieda umožňuje zadať nielen status kód a obsah odpovede, ale tiež vložiť metadáta do hlavičky.

```
@GET
@Produces(MediaType.TEXT_PLAIN)
@Path("predmet/{nazov}")
public Response getPredmet(@PathParam("nazov") String name) {

    if ("VSA".equals(name)) {
        return Response
            .status(Response.Status.OK)
            .entity("Informácie o predmete VSA")
            .build();
    }
}
```



```
// a.t.d.
return Response
    .status(Response.Status.NO_CONTENT)
    .build();
}
```

Výber metódy na spracovanie HTTP požiadavky

Pre konkrétnu URI a HTTP požiadavku, môže teda trieda poskytovať aj viacero metód, musia sa však odlišovať aspoň MIME-typom v anotáciách `@Produces`, resp. `@Consumes`.

V našom príklade môžeme pre koreňový zdroj okrem GET-metódy, ktorá vracia názov školy (ako jednoduchý text) implementovať aj druhú, ktorá vráti obrázok s logom školy.

```
@Path("skola")
public class SkolaResource {
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getInfo() {
        return "FEI STU";
    }

    @GET
    @Produces("image/*")
    public Response getLogo() {
        File f = new File("/home/edu/Pictures/feistu.jpg");
        if (!f.exists()) {
            throw new WebApplicationException(404);
        }
        String mt = new MimetypesFileTypeMap().getContentType(f);
        return Response.ok(f, mt).build();
    }
}
```

Pri výbere metódy na spracovanie konkrétnej HTTP požiadavky sa postupuje podľa nasledujúcich zásad:

- URI požiadavky sa porovná s URI vytvorenými na základe `@Path` anotácie triedy a metódy.
- HTTP metóda požiadavky sa porovná s anotáciou metódy zdroja (`@GET` `@POST`,...)
- Formát reprezentácie, ktorý požaduje klient (nachádza sa v hlavičke `Accept` požiadavky) porovná s MIME-typom v anotácii `@Produces`.
- Formát obsahu požiadavky (nachádza sa v hlavičke `Content-type`) sa porovná s MIME-typom v anotácii `@Consumes`.

Pozri [Declarative content negotiation v Oracle JAX-RS dokumentácii](#).

Ktorá z metód v našom príklade spracuje požiadavku GET na koreňový zdroj (či odpoveďou bude text alebo obrázok), závisí teda od toho, čo klient očakáva. Pozri cvičenie 7.5.

Poznámka: Metóda GET, ktorá vracia súbor (napr. obrázok) ako reprezentáciu zdroja, na to použije opäť triedu `Response`. Tá umožňuje serializovať súbor do obsahu správy a do hlavičky vložiť jeho formát, príp. ďalšie metadáta.

Implementáciu príkladu 2 ako webovej aplikácie pre server Glassfish 8 nájdete v projekte [SkolaRest](#).

Cvičenia

Cvičenie 7.1

Reimplementujte službu z projektu [SimpleRestApp](#) ako súčasť webovej aplikácie a otestujte ju. Postup:

- Vytvorte projekt web aplikácie pre **Glassfish8** a nazvite ho **SimpleRestWeb**. Preneste do neho implementáciu triedy zdroja **SimpleResource**. (Žiadne iné zmeny v projekte ani triede zdroja nie sú potrebné.)

- Spustíte projekt SimpleRestWeb, vývojové prostredie spustí aplikačný server s webovou aplikáciou a otvorí prehliadač.
- V prehliadači zadajte URI zdroja:
`http://localhost:8080/SimpleRestWeb/resources/message`
 Mali by ste dostať ako odpoveď "ziadny oznam".
- Otestujte službu pomocou klientskej aplikácie [SimpleRestClient](#).
Pozor. Najprv však treba v klientskej aplikácii upraviť **BASE_URI** na
`http://localhost:8080/SimpleRestWeb/resources`

Cvičenie 7.2

Pokračujte v projekte **SimpleRestWeb** z predchádzajúceho cvičenia. Ostráňte z triedy zdroja anotáciu `@Singleton` a presvedčte sa pomocou klientskej aplikácie [SimpleRestClient](#), že teraz nie je možné zmeniť text správy pomocou PUT.

Cvičenie 7.3 Využitie kontextu servletu

Ak nechceme/nemôžeme zabrániť (pomocou vhodnej anotácie), aby sa pre každú HTTP požiadavku vytvárala nová inštancia triedy zdroja, môžeme stav zdroja (aktuálnu správu) uložiť v kontexte servletu REST služby. Tento kontext sa vytvára len raz pri spustení web aplikácie. Referenciu na neho môžeme injektovať ako argument metódy zdroja pomocou anotácie `@Context ServletContext`.

Upravte implementáciu triedy zdroja v projekte **SimpleRestWeb** a otestujte ju klientom.

```
@Path("message")
public class SimpleResource {
    private String oznam = "ziadny oznam";

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getText( @Context ServletContext ctx) {
        return ctx.getAttribute("sprava");
    }

    @PUT
    @Consumes(MediaType.TEXT_PLAIN)
    public void putMessage(@Context ServletContext ctx, String content) {
        ctx.setAttribute("sprava", content);
    }
}
```

Cvičenie 7.4 Prístup web aplikácie k súborom na serveri

GET metóda `getLogo` v príklade 2 (projekt [SkolaRest](#)), číta obrázok zo súboru na lokálnom súborovom systéme. Pri nasadzovaní služby na aplikačný server treba s ňou preniesť aj súbor s obrázkom. Preto je potrebné umiestniť obrázok do projektu webovej aplikácie. V čase implementácie síce nepoznáme absolútnu cestu koreňového adresára projektu na aplikačnom serveri, ale kontext REST servleta ju umožňuje získať.

```
@GET
@Produces("image/*")
public Response getLogo( @Context ServletContext ctx ) {
    File f = new File(ctx.getRealPath("feistu.jpg"));
}
```

Umiestnite súbor s obrázkom priamo do koreňového adresára (je tam aj `index.html`), upravte implementáciu metódy `getLogo` a otestujte ju.

Cvičenie 7.5 Výber metódy pre spracovanie požiadavky

V príklade 2 (projekt [SkolaRest](#)) trieda zdroja poskytuje dve GET metódy pre koreňové URI `skola`, jedna vracia textovú reprezentáciu a druhá obrázok.

- A. Zistíte, ktorá metóda spracuje GET požiadavku na koreňové URI odoslanú z prehliadača. T.j. či dostanete textovú reprezentáciu alebo obrázok. Skúste rôzne prehliadače, príp. iné klientské aplikácie.

- B. V anotácii metódy zdroja `getInfo()` zmeňte formát z `TEXT_PLAIN` na `TEXT_HTML` a klientom opäť overte, ktorú reprezentáciu dostanete teraz.

Vysvetlenie: Pri výbere metódy pre spracovanie požiadavky sa zohľadňuje hodnota poľa `accept` v hlavičke požiadavky. Pre prístup k hlavičke pozri cvičenie 7.9

Cvičenie 7.6 Relatívna URI zdroja definovaná regulárnym výrazom

Parametrizovaný fragment URI v anotácii `@Path` môže obsahovať regulárny výraz, ktorý definuje platné hodnoty fragmentu. Do projektu [SkolaRest](#) pridajte GET metódu, ktorá vracia informácie o triedach gymnázia.

```
@Path("trieda/{nazov: [1-4][A-C]}")
@Produces(MediaType.TEXT_PLAIN)
public String getTrieda(@PathParam("nazov") String trieda) {
    return "Informácie o triede " + trieda;
}
```

Presvedčte sa, že platné sú len názvy tried odpovedajúce regulárnemu výrazu `[1-4][A-C]`. Zistite, akú odpoveď vráti GET požiadavka na triedu s neplatným názvom.

Cvičenie 7.7 Číslo ako relatívna URI zdroja

Ak je parametrizovaný fragment URI v anotácii `@Path`, číslo, môžeme dať argumentu metódy číselný typ. Do projektu [SkolaRest](#) pridajte GET metódu, ktorá vracia informácie o ročníkoch, pričom platné ročníky sú čísla 1-9.

```
@Path("{rocnik}")
@Produces(MediaType.TEXT_PLAIN)
public String getTrieda(@PathParam("rocnik") int n) {
    if (n>0 && n<10) {
        return "Informacie o rocniku " + n;
    }
    return "Neplatny rocnik " + n;
}
```

Presvedčte sa, že platné URI sú len tie, v ktorých je fragment `rocnik` číslo 1-9.

Všimnite si tiež, že konverzia na číslo sa urobí automaticky pri extrakcii fragmentu z URI a zistíte, akú HTTP odpoveď vráti požiadavka, ak ročník v URI nie je číslo.

Cvičenie 7.8 Parametre dotazu

Metódy zdroja môžu získať a použiť aj parametre HTTP požiadavky. Hodnota parametra je injektovaná ako argument metódy anotáciou `@QueryParam("meno")`, kde `meno` je identifikátor parametra, ako ilustruje jednoduchý príklad služby:

```
@Path("pozdrav")
public class PozdravResource {
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getText(@QueryParam("meno") String name) {
        return "Dobré ráno " + name;
    }
}
```

Implementujte službu a overte si odpoveď na GET požiadavku pre URI:

`<ip-servera>/<aplikacia>/resources/pozdrav?meno=Fero`

Doplňte deklaráciu metódy o definíciu náhradnej hodnoty pre parameter požiadavky.

```
public String getText(@DefaultValue("neznámy") @QueryParam("meno") String name)
```

Presvedčte sa, že odpoveď na GET požiadavku bez parametra `meno` obsahuje náhradnú hodnotu.

Cvičenie 7.9 Hlavička dotazu

Pomocou anotácie `@Context` je možné injektovať ako argument metódy zdroja tiež objekt triedy `HttpHeaders`. Prostredníctvom neho má metóda prístup ku všetkým údajom v hlavičke HTTP požiadavky.

Implementujte nasledujúcu GET metódu a presvedčte sa, že vráti obsah hlavičky dotazu.

```
@GET
@Produces("text/html")
public String getHeader(@Context HttpHeaders hh) {
    MultivaluedMap<String, String> headerParams = hh.getRequestHeaders();
    String resp = "";
    for(String key : headerParams.keySet()) {
        resp = resp + key + " = " + headerParams.getFirst(key) + "<br/>";
    }
    return resp;
}
```